

FULLSTACK_WEBDEV

- <https://github.com/Bharat2044/100xDevs-Cohort3-WebDev-and-Devops/blob/main/Lecture%20Slides.md>
- <https://app.100xdevs.com/courses/14>
- 2-3 Years Work => Muscle Memory forever
- DevOps+ MERN => SDE2/ TECH LEAD/ STAFF ENGINEER LEVEL
-

Table of Contents

- [FULLSTACK_WEBDEV](#)
 - [Table of Contents](#)
 - [1 Orientation](#)
 - [Course Overview](#)
 - [Student Level Classifications](#)
 - [Development Environment](#)
 - [HTML Fundamentals](#)
 - [CSS Fundamentals](#)
 - [Flexbox Layout System](#)
 - [CSS Classes and IDs](#)
 - [Color Systems and RGB Values](#)
 - [CSS Margin and Padding Values](#)
 - [Border Properties](#)
 - [Advanced CSS Concepts](#)
 - [Browser Tools and Development](#)
 - [Key Concepts Summary](#)
 - [Next Steps and Advanced Topics](#)
 - [2 Javascript Basics](#)
 - [Javascript - The basics](#)
 - [Properties of JS](#)
 - [Syntax of Javascript](#)
 - [Complex types](#)
 - [3 Asynchronous Javascript, Callbacks and more](#)
 - [Intro](#)
 - [Normal functions in JS](#)
 - [Synchronous code](#)
 - [I/O heavy operations](#)
 - [I/O bound tasks vs CPU bound tasks](#)
 - [Doing I/O bound tasks in the real world](#)
 - [Functional arguments](#)
 - [Asynchronous code, callbacks](#)
 - [JS Architecture for async code](#)
 - [4 Promises and async, await](#)
 - [Classes in JS](#)

- [Inheritance in classes](#)
- [Some more classes](#)
- [Promise class](#)
- [Callback hell](#)
- [Async await syntax](#)
- [Defining your own async function](#)
- [err first callback vs rejects in promises](#)
- [Assignment](#)

1 Orientation

Course Overview

- Goal

Make you incredibly solid at building and deploying Web apps.

- Cohort Projects

1. <https://github.com/code100x/cms/> - Led by hsingh
2. <https://github.com/code100x/daily-code/> - Led by Sargam
3. <https://github.com/code100x/job-board> - Led by Vineet
4. <https://github.com/code100x/algorithmic-arena/pull/10> - Led by Nimit
5. Framer like app - Up for grabs

- Avoid the Spikes Strategy

1. We will go slow, today only focussing on HTML/CSS/JS
2. Any topic that you feel is missing / you want we will do polls from time to time we can add
3. Go slow but stay consistent, avoid the spikes

- What would I do if I was you?

1. Get to know people in discord, try to find a group of 3-4 motivated people
2. Look at all the success stories here and how they got a job - <https://www.youtube.com/@100xDevs-n1w>
3. Repeated advice - GSoC is the best internship you can do. Already start prepping for it competition is high. Tell me how can I help here we're hoping for ~50 GSoC selections from the cohort this year.
Good video to watch - <https://www.youtube.com/watch?v=OmzCvb-QBak>

- Sprint Options

If things are going too slow you can:

1. Participate in projects/hackathons
2. Sprint through Cohort 2.0

Student Level Classifications

- Complete Beginner (very basic/no coding before)

- Things will be hard. You should spend ~20 hours/week at least on Cohort material, assignments, other content covering what we're doing in the Cohort. Don't slack off
- Intermediate (Done basic JS, HTML and some DSA)
 - You should be for the first ~10 weeks or so. We see major drop off after the first major project
 - Try chewing some glass after week 10, that is where you will actually learn things that differentiate you from a typical MERN developer
- Advanced (Know MERN decent/well)
 - Feel free to drop off for the first few weeks. I'll see you on Github. Participate in the Superteam hackathon, aim for GSoC this year. Already start looking at orgs

Development Environment

- VSCode Installation

Download from here - <https://code.visualstudio.com/>

- Good Things to Hash Out
 1. Installing VSCode locally
 2. HTML/CSS
 3. Cloning the landing page of a simple website
- Step 1: Creating a Simple HTML File Locally
 1. Create a folder on your machine called **website**
 2. Add a **index.html** file inside it
 3. Write the following code in it

```
<html>
  <title>
    Visual Studio Code - Code Editor
  </title>
</html>
```

Now open this in your browser

HTML Fundamentals

HTML stands for HyperText Markup Language. It is the foundation of building **unstyled** websites.

- not case sensitive
- sandwich code language
- **/** : root directory of website
- **
** : eg of autoclosing tag

- Step 2: Tags

1. div, span
2. head
3. body
4. h1, h2, h3, h4, h5, h6
5. b, i, u
6. a
7. img
8. input
9. button
10. br

- HTML Tag Structure

```
html - root element
├─ head - metadata container
│   ├── title - page title
│   └── style - internal styles
└─ body - main content container
    ├── div - block-level container
    ├── span - inline container
    ├── h1-h6 - headings (largest to smallest)
    ├── b, i, u - bold, italic, underline
    ├── a href - links to navigate
    ├── img src - embed images
    ├── input type - user input fields
    ├── button - clickable buttons
    └── br - line breaks
```

- Step 3: Building a Basic HTML Page

```
<html>
  <title>
    Visual Studio Code - Code Editor
  </title>
</html>
<body>
  <div>
    <span>Visual Studio Code</span>
    <a href="/">Docs</a>
    <a href="/">Updates</a>
    <a href="/">Blog</a>
    <a href="/">API</a>
    <a href="/">Extensions</a>
    <a href="/">FAQs</a>
    <a href="/">Learn</a>

    <input type="text" placeholder="Search Docs">
```

```
        <button>Download</button>
    </div>
<br/>

    <div>
        <a href="/">Version 1.82</a> is now available! Read about the new
features and fixes from July.
    </div>

    <br/>
</body>
```

CSS Fundamentals

CSS stands for Cascading Style Sheets. It is used to style our applications.

- CSS Integration Methods

You can add CSS to your HTML app by using:

1. The `style` attribute (inline styles)
2. In an external css file

- Approach 1: Inline Styles

Try updating the `body` tag in the last style as follows:

```
<body style="background-color: black;">
... rest of the code
</body>
```

- Approach 2: External Styles

1. Add a new file called `index.css`
2. Add the following code in it

```
body {
    background-color: black;
}
```

Priority: Inline styles >> External style

Update `index.html`:

```
<html>
  <title>
    Visual Studio Code - Code Editor
```

```
    </title>
    <link rel="stylesheet" href="index.css">
</html>
<body>
... rest of the code
</body>
```

- Common Style Attributes
- **color**: Sets the text color
- **background-color**: Sets the background color
- **font-size**: Sets the size of the text
- **margin**: Sets the outer space around an element
- **padding**: Sets the inner space within an element
- **border**: Sets the border around an element

Flexbox Layout System

Flexbox is a CSS layout model designed to help with the arrangement of items within a container.

- Basic Flexbox Implementation

Update the website to the following:

```
<html>
  <title>
    Visual Studio Code - Code Editor
  </title>
</html>
<body>
  <div style="display: flex;">
    <div>Visual Studio Code</div>
    <a href="/">Docs</a>
    <a href="/">Updates</a>
    <a href="/">Blog</a>
    <a href="/">API</a>
    <a href="/">Extensions</a>
    <a href="/">FAQs</a>
    <a href="/">Learn</a>
  </div>
  <div>
    <input type="text" placeholder="Search Docs">
    <button>Download</button>
  </div>
  <br/>

  <div>
```

```

    <a href="/">Version 1.82</a> is now available! Read about the new
    features and fixes from July.
  </div>

  <br/>
</body>

```

Notice that the elements are positioned right next to each other even though **Visual Studio code** is inside a **div**.

- Justify Content Property

Try experimenting with the **justify-content** property:

```

<html>
  <title>
    Visual Studio Code - Code Editor
  </title>
</html>
<body>
  <div style="display: flex; justify-content: space-between;">
    <div>Visual Studio Code</div>
    <a href="/">Docs</a>
    <a href="/">Updates</a>
    <a href="/">Blog</a>
    <a href="/">API</a>
    <a href="/">Extensions</a>
    <a href="/">FAQs</a>
    <a href="/">Learn</a>
  </div>
  <div>
    <input type="text" placeholder="Search Docs">
    <button>Download</button>
  </div>
  <br/>

  <div>
    <a href="/">Version 1.82</a> is now available! Read about the new
    features and fixes from July.
  </div>

  <br/>
</body>

```

- Flexbox Properties from Note s
- Display and Justify Content

```

display: flex;          /* Enable flexbox layout */
justify-content: flex-start; /* Align items to start */

```

```
justify-content: flex-end;      /* Align items to end */
justify-content: center;        /* Center items */
justify-content: space-between; /* Space items with gaps between */
justify-content: space-around;  /* Space items with equal space around */
justify-content: space-evenly;  /* Space items with even distribution */
```

```
display: flex;                  /* Enable flexbox layout */

/* justify-content property controls main axis alignment */

justify-content: flex-start;    /* Align items to start */
/*


[A] [B] [C]


*/

justify-content: flex-end;      /* Align items to end */
/*


[A] [B] [C]


*/

justify-content: center;        /* Center items */
/*


[A] [B] [C]


*/

justify-content: space-between; /* Space items with gaps between */
/*


[A] [B] [C]


*/

justify-content: space-around;  /* Space items with equal space around */
/*


[A] [B] [C]


*/

justify-content: space-evenly;  /* Space items with even distribution */
/*


[A] [B] [C]


*/
```


- Advanced Flexbox Example

```
<html>
</html>
<body>
  <header>
  </header>
  <section>
    <div style="border-width: thick; border-style: solid; display:
flex; justify-content: space-between; margin-left: 200px; margin-right:
200px;">
      <div style="background: red;">
        <h1>Code with GitHub Copilot</h1>
        <h2>Write code faster and smarter with GitHub Copilot, your
AI pair programmer.</h2>
        Try GitHub Copilot free for 30 days
        Completions present suggestions automatically to help you
code more efficiently.

        Copilot Chat understands the context of your code,
workspace, extensions, settings, and more.

        Inline Chat enables you to iteratively generate edits and
get answers to quick questions, directly on your code.
      </div>
      <div style="background: green;">
        
      </div>
    </div>
  </section>
  <footer>
  </footer>
</body>
```

CSS Classes and IDs

In CSS, classes and IDs are used as selectors to apply styles to HTML elements. They help in targeting specific elements for styling and can be used to enhance the modularity and reusability of CSS code.

- Classes vs IDs Usage
- Classes (.) - Reusable Styles

```
<span class="toplink">Docs</span>
<span class="toplink">Updates</span>
```

```
.toplink {  
  color: gray;  
}
```

- IDs (#) - Unique Elements

```
<div id="header">Content</div>
```

```
#header {  
  background-color: blue;  
}
```

Color Systems and RGB Values

- You can create any color with combination of Red, Blue, Green
- RGB is a color model used to represent colors in digital displays
- r (0 - 255), g (0 - 255), b (0 - 255) [min- max values]
- RGB Color Model

```
rgb(255, 0, 0)      /* Red */  
rgb(0, 255, 0)     /* Green */  
rgb(0, 0, 255)     /* Blue */  
rgb(255, 255, 0)   /* Yellow */  
rgb(255, 0, 255)   /* Purple */  
rgb(0, 255, 255)   /* Cyan */
```

- RGBA with Transparency

```
rgba(255, 0, 0, 0.5) /* Semi-transparent red */  
rgba(0, 0, 0, 0.3)  /* Semi-transparent black */
```

- Hex Color Codes

```
#FF0000 /* Red */  
#00FF00 /* Green */  
#0000FF /* Blue */  
#FFFF00 /* Yellow */
```

CSS Margin and Padding Values

```
margin: 10px;           /* All sides 10px */
margin: 10px 20px;      /* Top/bottom 10px, left/right 20px */
margin: 10px 20px 30px; /* Top 10px, left/right 20px, bottom 30px */
margin: 10px 20px 30px 40px; /* Top, right, bottom, left */

padding: 15px;          /* All sides 15px */
padding: 10px 20px;     /* Top/bottom 10px, left/right 20px */
```

Border Properties

```
border: 1px solid black; /* Width style color */
border-width: thick;     /* Border thickness */
border-style: solid;     /* Border style */
border-color: red;       /* Border color */
```

Advanced CSS Concepts

- Font and Text Properties

```
font-size: 16px;        /* Text size */
font-weight: bold;      /* Text weight */
text-align: center;     /* Text alignment */
text-decoration: underline; /* Text decoration */
```

- Background Properties

```
background-color: black; /* Background color */
background: red;         /* Shorthand background */
```

- Element Positioning and Layout

```
margin-left: 200px;     /* Left margin */
margin-right: 200px;    /* Right margin */
width: 800px;           /* Element width */
```

- CSS Selectors Hierarchy

```
Element Selectors:
|— Tag selectors: div, span, body
```

```
|— Class selectors: .classname  
|— ID selectors: #idname  
|— Attribute selectors: [type="text"]
```

Browser Tools and Development

- Google Chrome Developer Tools
- Right-click → Inspect Element
- F12 or Ctrl+Shift+I
- Elements tab for HTML structure
- Styles tab for CSS properties
- File Structure Organization

```
website/  
|— index.html  
|— index.css  
|— images/  
|— js/ (for future JavaScript)
```

- HTML Document Structure Best Practices

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Page Title</title>  
  <style>  
    /* Internal styles */  
  </style>  
  <link rel="stylesheet" href="index.css">  
</head>  
<body>  
  <!-- Page content -->  
</body>  
</html>
```

Key Concepts Summary

- HTML Core Concepts
- HyperText Markup Language foundation
- Tag-based structure with opening/closing tags
- Semantic elements for content organization

- Attributes provide additional information
 - CSS Core Concepts
 - Cascading Style Sheets for presentation
 - Selectors target HTML elements
 - Properties define visual characteristics
 - Values specify property settings
 - Flexbox Layout Model
 - Modern CSS layout system
 - Container and item relationship
 - Main axis and cross axis alignment
 - Flexible item sizing and positioning
 - Development Workflow
1. Create HTML structure
 2. Add CSS styling
 3. Test in browser
 4. Iterate and refine
 5. Validate code quality

Next Steps and Advanced Topics

- Upcoming Technologies
- JavaScript for interactivity
- Responsive design principles
- CSS Grid layout system
- Modern development tools
- Project Milestones
- Clone simple website landing pages
- Build responsive layouts
- Implement interactive features
- Deploy to web hosting platforms
- Learning Resources
- 100xDevs community Discord

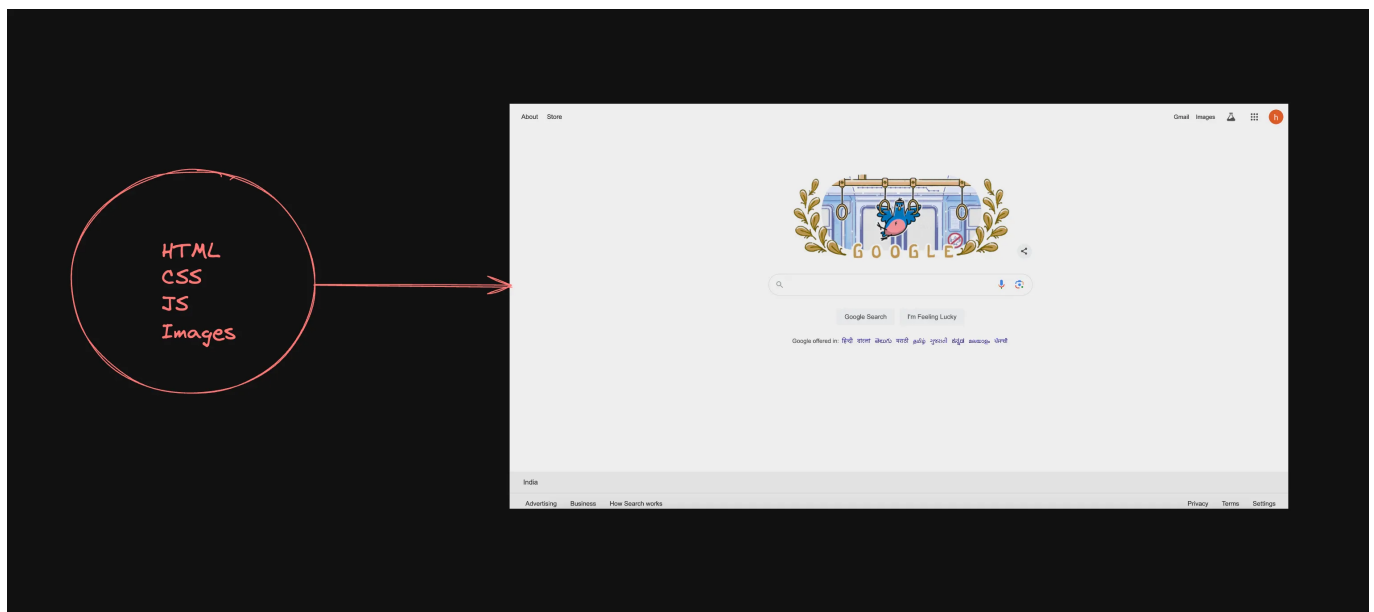
- YouTube success stories and tutorials
- GitHub project repositories
- GSoC preparation materials

2 Javascript Basics

Javascript - The basics

- Web development

Web development involves writing a lot of HTML, CSS and JS code. Historically (and even today to some extent), browsers could only understand HTML, CSS and JS. Any website that you see, is a bunch of HTML, CSS and JS files along with some assets (images, videos etc)



- Facts/Callouts

React, NextJS are frameworks . They compile down to HTML, CSS, JS in the end. That is what your browser understands. When you run your C++ code on leetcode , it does not run on your browser/machine. It runs somewhere else. Your browser can't (almost) compile and run C++ code. If someone asks — What all languages can your browser interpret, the answer is HTML, CSS, JS and WebAssembly. It can, technically, run C++/Rust code that is compiled down to Wasm

- Before we proceed, do one of the following

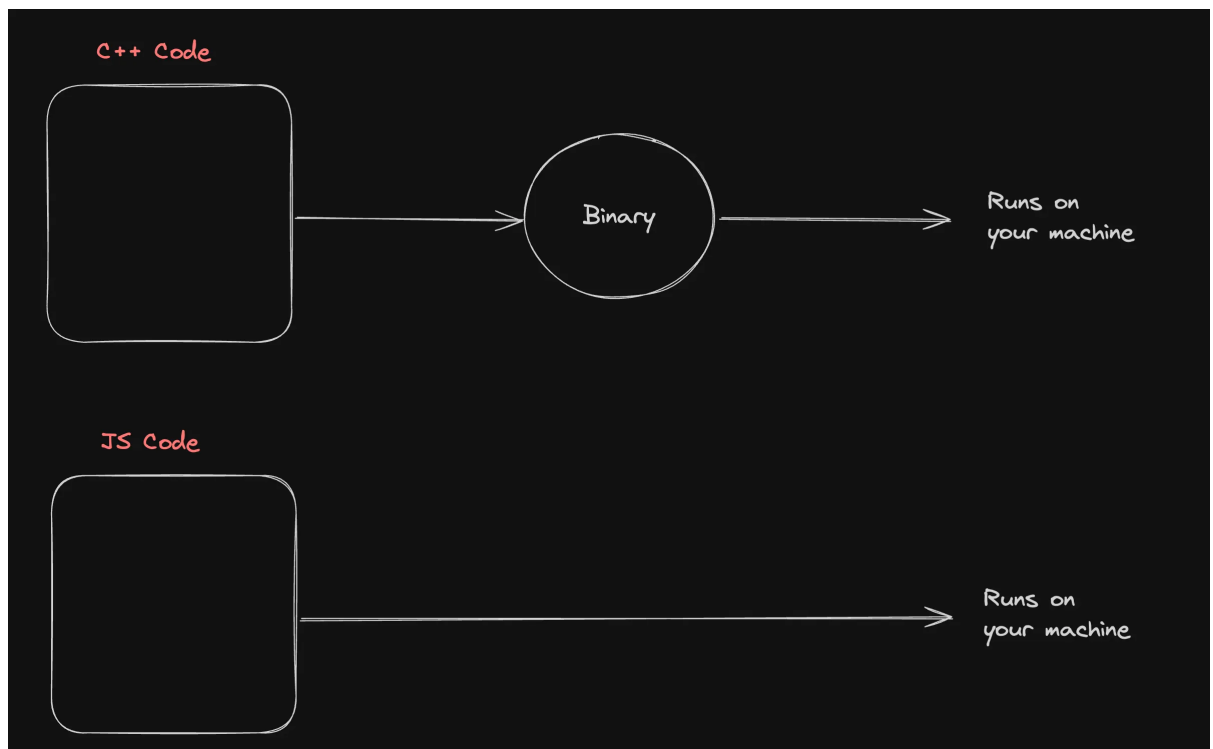
Create an account on replit Install Node.js locally Keep your browser console open for testing locally

Properties of JS

Every language comes with it's unique set of features.

- Javascript has the following -
 1. Interpreted JavaScript is an interpreted language, meaning it's executed line-by-line at runtime by the JavaScript engine in the browser or server environment, rather than being compiled into

machine code beforehand.



Upsides - There is one less step to do before running your code Downsides - Performance Overhead: More prone to runtime errors

- 2. Dynamically Typed Variables in JavaScript are not bound to a specific data type. Types are determined at runtime and can change as the program executes C++ Code (won't compile)

```
#include <iostream>

int main() {
    int a = 1;
    a = "hello";
    a = true;
}
```

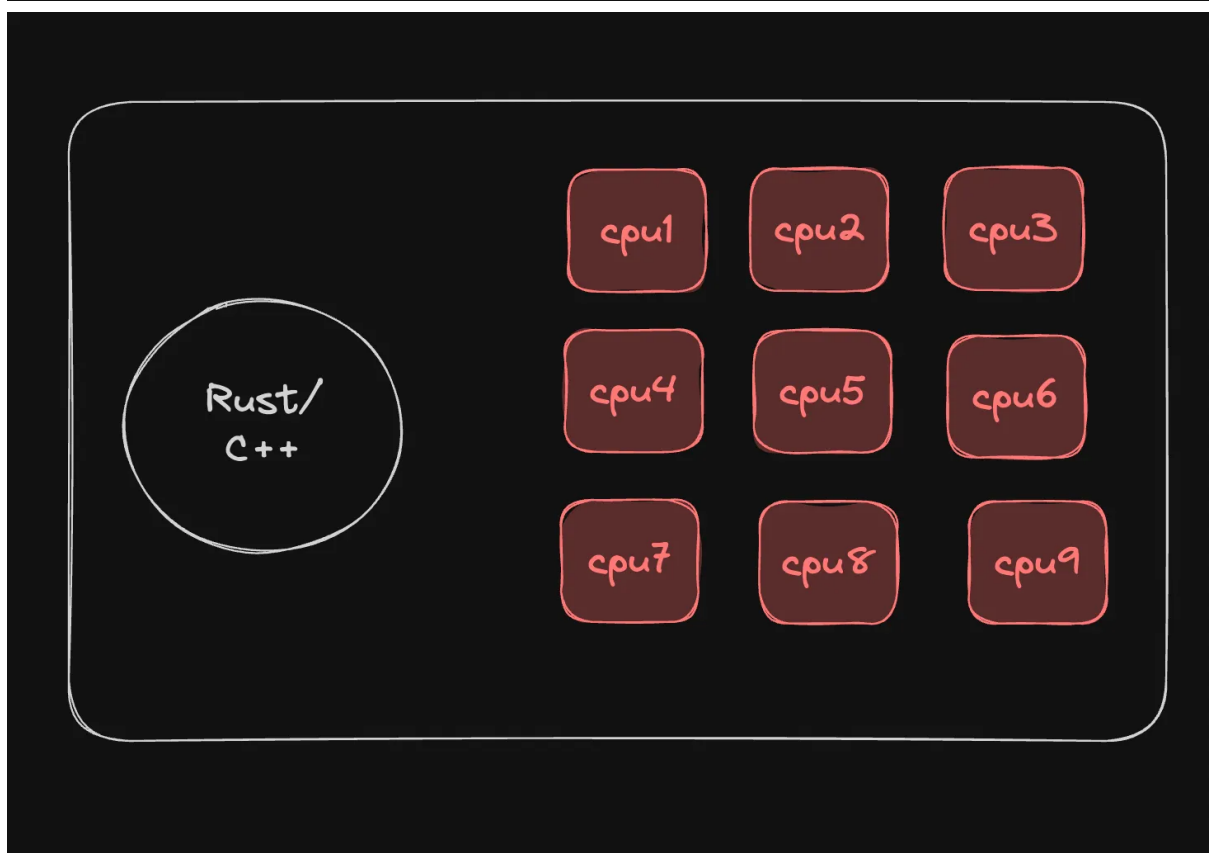
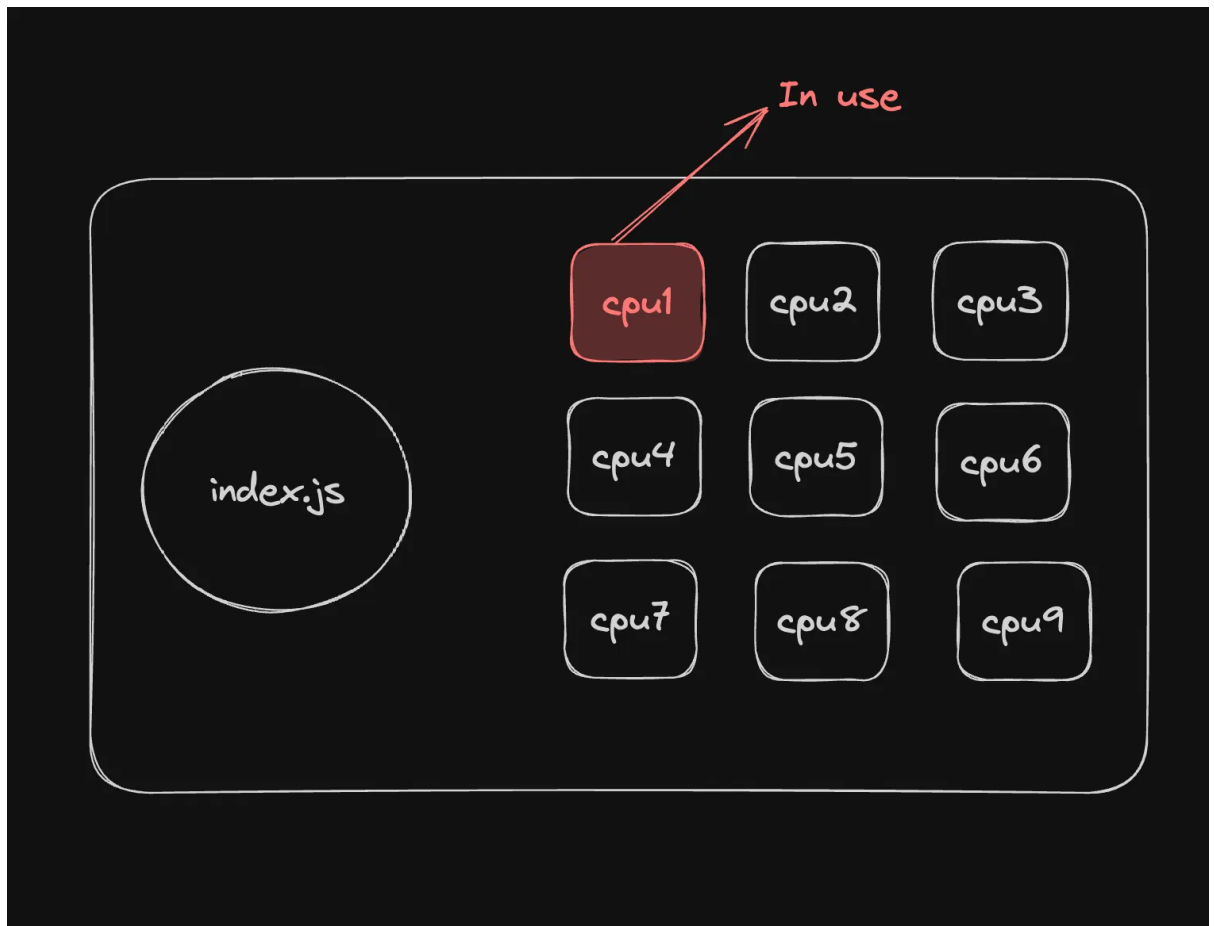
JS Code (will compile)

```
var a = 1;
a = "harkirat";
a = true;

console.log(a)
```

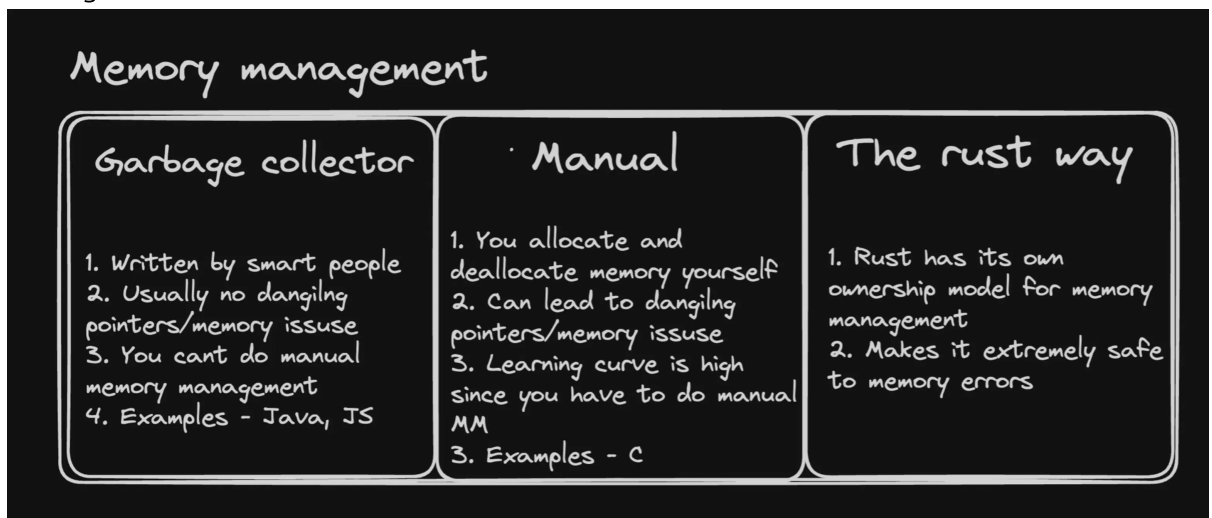
- cpp code is statically typed
- js code is dynamically typed

- Rust
 - do ownership checks
 - compiler very strict
 - extreme memory safe
 - extreme slow compile
- 3. Single threaded JavaScript executes code in a single-threaded environment, meaning it processes one task at a time. We will dive deeper into this next week.



- even JS infinite loop (most expensive operation :0) can utilize max only 1CPU !!!
- (do htop ;0)
 - at htop, nodejs process => 98% CPU
 - it uses ~100%CPU out of $100\% * \text{number of CPUs total}$

- eg: i have 20 CPU Cores, uses 98% nodejs app, from $20 \times 100\%$ total = 2000% total CPU space
- Rust/ C++(Multithreaded) can go to 200%, 300% etc.
- ffmpeg(c++) go upto 300% / even use full CPUs
- to reduce JS Runtime errors, TS was introduced
 - TS(Like C++) =(compile)> JS => Run
 - it reduces disadvantages
- companies go from JS to Rust, Golang for
 - faster Run
 - Max. Safety
- 4. Garbage collected



JavaScript automatically manages memory allocation and deallocation through garbage collection, which helps prevent memory leaks by automatically reclaiming memory used by objects no longer in use.

- Conclusion Is JS a good language? Yes and no. It is beginner friendly, but has a lot of performance overhead. Bun is trying to solve for a lot of this, but there's a long way to go before JS can compete with languages like C++/Rust

Syntax of Javascript

- 1. Variables Variables are used to store data. In JavaScript, you declare variables using var, let, or const.

```
let name = "John";    // Variable that can be reassigned
const age = 30;       // Constant variable that cannot be reassigned
var isStudent = true; // Older way to declare variables, function-scoped
```

- Assignment Create a variable for each of the following: your favorite color, your height in centimeters, and whether you like pizza. Use appropriate variable declarations (let, const, or var). Try logging it using console.log
- 2. Data types

```
let number = 42;           // Number
let string = "Hello World"; // String
let isActive = false;      // Boolean
let numbers = [1, 2, 3];   // Array
```

- 3. Operators

```
let sum = 10 + 5;           // Arithmetic operator
let isEqual = (10 === 10); // Comparison operator
let isTrue = (true && false); // Logical operator
```

- 4. Functions

```
// Function declaration
function greet(name) {
    return "Hello, " + name;
}

// Function call
let message = greet("John"); // "Hello, John"
```

- Assignment #1 Write a function sum that finds the sum of two numbers. Side quest - Try passing in a string instead of a number and see what happens? Assignment #2 Write a function called canVote that returns true or false if the age of a user is > 18
- 5. If/Else

```
if (age >= 18) {
    console.log("You are an adult.");
} else {
    console.log("You are a minor.");
}
```

- Assignment Write an if/else statement that checks if a number is even or odd. If it's even, print "The number is even." Otherwise, print "The number is odd."
- 6. Loops

```
// For loop
for (let i = 0; i < 5; i++) {
    console.log(i); // Outputs 0 to 4
}

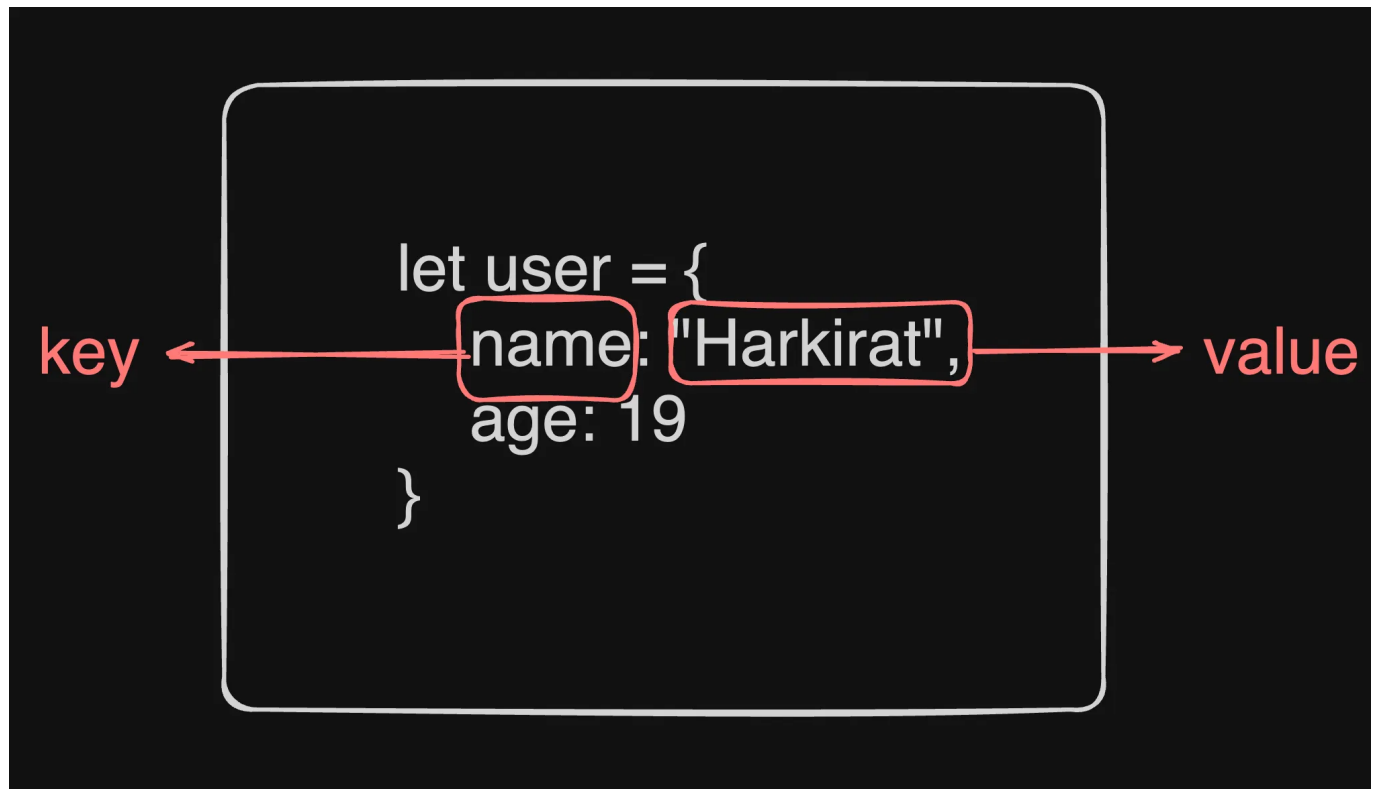
// While loop
let j = 0;
```

```
while (j < 5) {  
  console.log(j); // Outputs 0 to 4  
  j++;  
}
```

- Assignment Write a function called sum that finds the sum from 1 to a number

Complex types

- Objects An object in JavaScript is a collection of key-value pairs, where each key is a string and each value can be any valid JavaScript data type, including another object.



```
let user = {  
  name: "Harkirat",  
  age: 19  
}  
  
console.log("Harkirats age is " + user.age);
```

- Assignment #1 Write a function that takes a user as an input and greets them with their name and age
- Assignment #2 Write a function that takes a new object as input which has name , age and gender and greets the user with their gender (Hi Mr/Mrs/Others harkirat, your age is 21)
- Assignment #3 Also tell the user if they are legal to vote or not
- Arrays Arrays let you group data together

```
const users = ["harkirat", "raman", "diljeet"];  
const totalUsers = users.length;
```

```
const firstUser = users[0];
```

- Assignment Write a function that takes an array of numbers as input, and returns a new array with only even values. Read about filter in JS
- Array of Objects We can have more complex objects, for example an array of objects

```
const users = [{
  name: "Harkirat",
  age: 21
}, {
  name: "raman",
  age: 22
}]

const user1 = users[0]
const user1Age = users[0].age
```

- Assignment Write a function that takes an array of users as inputs and returns only the users who are more than 18 years old
- Object of Objects We can have an even more complex object (object of objects)

```
const user1 = {
  name: "harkirat",
  age: 19,
  address: {
    city: "Delhi",
    country: "India",
    address: "1122 DLF"
  }
}

const city = user1.address.city;
```

- Assignment Create a function that takes an array of objects as input, and returns the users whose age > 18 and are male

3 Asynchronous Javascript, Callbacks and more

Intro

- Goal of todays class:
 - I/O tasks
 - Callbacks
 - Functional arguments
 - Async vs Sync codeEvent loops, callback queues, JS

- Goal of tomorrows class
- Async await, Promises
- Practising async JS

Hopefully, by the end of the class, you are able to understand the following code:

- Functional arguments

```
function sum(a, b) {  
  return a + b;  
}  
  
function multiply(a, b) {  
  return a * b;  
}  
  
function subtract(a, b) {  
  return a - b;  
}  
  
function divide(a, b) {  
  return a / b;  
}  
  
function doOperation(a, b, op) {  
  return op(a, b)  
}  
  
console.log(doOperation(1, 2, sum))
```

- Callbacks

```
const fs = require("fs");  
  
fs.readFile("a.txt", "utf-8", function (err, contents) {  
  console.log(contents);  
});
```

Normal functions in JS

The way to write functions in JS is as follows - Find sum of two numbers

```
function sum(a, b) {  
  return a + b;  
}  
  
let ans = sum(2, 3)  
console.log(sum);
```

Find sum from 1 to a number

```
function sum(n) {  
  let ans = 0;  
  for (let i = 1; i <= n; i++) {  
    ans = ans + i  
  }  
  return ans;  
}  
  
const ans = sum(100);  
console.log(ans);
```

Synchronous code

Synchronous code is executed line by line, in the order it's written. Each operation waits for the previous one to complete before moving on to the next one.

For example

```
function sum(n) {  
  let ans = 0;  
  for (let i = 1; i <= n; i++) {  
    ans = ans + i  
  }  
  return ans;  
}  
  
const ans1 = sum(100);  
console.log(ans1);  
const ans2 = sum(1000);  
console.log(ans2);  
const ans3 = sum(10000);  
console.log(ans3);
```

I/O heavy operations

I/O (Input/Output) heavy operations refer to tasks in a computer program that involve a lot of data transfer between the program and external systems or devices. These operations usually require waiting for data to be read from or written to sources like disks, networks, databases, or other external devices, which can be time-consuming compared to in-memory computations.

- Examples of I/O Heavy Operations:
 - Reading a file
 - Starting a clock
 - HTTP Requests

💡 We're going to introduce imports/requires next. A require statement lets you import code/functions export from another file/module.

Let's try to write code to do an I/O heavy operation -

- Open repl.it
- Create a file in there (a.txt) with some text inside
- Write the code to read a file synchronously

```
const fs = require("fs");

const contents = fs.readFileSync("a.txt", "utf-8");
console.log(contents);
```

- Create another file (b.txt)
- Write the code to read the other file synchronously

```
const fs = require("fs");

const contents = fs.readFileSync("a.txt", "utf-8");
console.log(contents);

const contents2 = fs.readFileSync("b.txt", "utf-8");
console.log(contents2);
```

💡 What is wrong in this code above?

I/O bound tasks vs CPU bound tasks

- CPU bound tasks

CPU-bound tasks are operations that are limited by the speed and power of the CPU. These tasks require significant computation and processing power, meaning that the performance bottleneck is the CPU itself.

```
let ans = 0;
for (let i = 1; i <= 1000000; i++) {
  ans = ans + i
}
console.log(ans);
```

💡 A real world example of a CPU intensive task is running for 3 miles. Your legs/brain have to constantly be engaged for 3 miles while you run.

- I/O bound tasks

I/O-bound tasks are operations that are limited by the system's input/output capabilities, such as disk I/O, network I/O, or any other form of data transfer. These tasks spend most of their time waiting for I/O operations to complete.

```
const fs = require("fs");

const contents = fs.readFileSync("a.txt", "utf-8");
console.log(contents);
```

💡 A real world example of an I/O bound task would be Boiling water. I don't have to do much, I just have to put the water on the kettle, and my brain can be occupied elsewhere.

Doing I/O bound tasks in the real world

What if you were tasked with doing 3 things

- Boil some water.
- Do some laundry
- Send a package via mail

Would you do these

- One by one (synchronously)
- Context switch between them (Concurrently)
- Start all 3 tasks together, and wait for them to finish. The first one that finishes gets catered to first.

💡 Good talk - Concurrency is not parallelism - <https://www.youtube.com/watch?v=oV9rvDlIkEg>

- Synchronously (One by one)

```
const fs = require("fs");

const contents = fs.readFileSync("a.txt", "utf-8");
console.log(contents);

const contents2 = fs.readFileSync("b.txt", "utf-8");
console.log(contents2);

const contents3 = fs.readFileSync("b.txt", "utf-8");
console.log(contents3);
```

- Start all 3 tasks together, and wait for them to finish.

```
const fs = require("fs");

fs.readFile("a.txt", "utf-8", function (err, contents) {
  console.log(contents);
});
```

```
fs.readFile("b.txt", "utf-8", function (err, contents) {
  console.log(contents);
});

fs.readFile("a.txt", "utf-8", function (err, contents) {
  console.log(contents);
});
```

Functional arguments

Write a calculator program that adds, subtracts, multiplies, divides two arguments.

- Approach #1
- Calling the respective function

```
function sum(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

function subtract(a, b) {
  return a - b;
}

function divide(a, b) {
  return a / b;
}

function doOperation(a, b, op) {
  return op(a, b)
}

console.log(sum(1, 2))
```

- Approach #2
- Passing in what needs to be done as an argument.

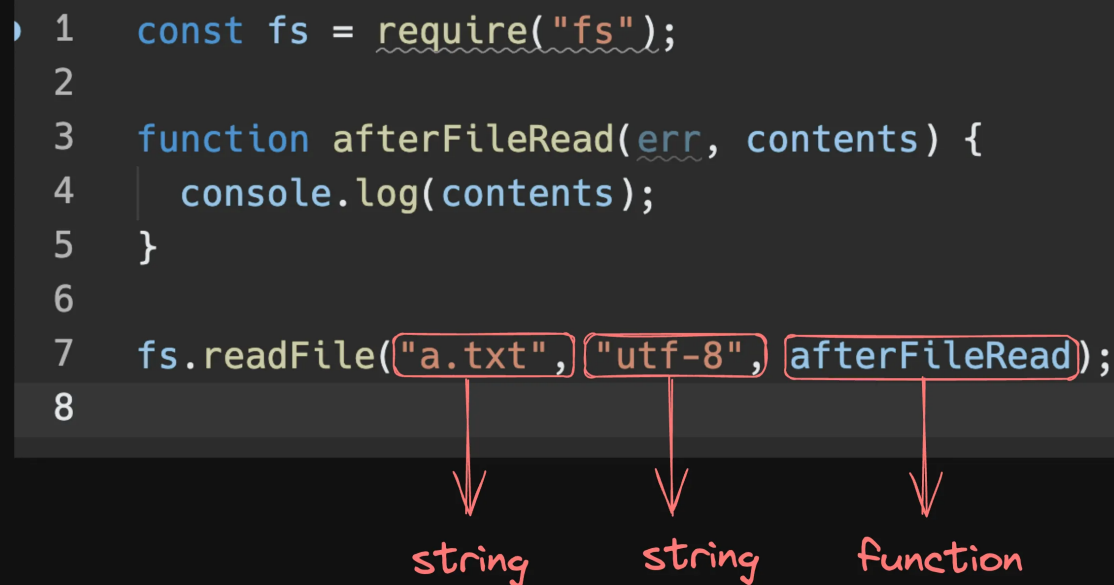
```
function sum(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}
```

```
function subtract(a, b) {  
  return a - b;  
}  
  
function divide(a, b) {  
  return a / b;  
}  
  
function doOperation(a, b, op) {  
  return op(a, b)  
}  
  
console.log(doOperation(1, 2, sum))
```

Asynchronous code, callbacks

Let's look at the code to read from a file asynchronously. Here, we pass in a function as an argument. This function is called a callback since the function gets called back when the file is read



```
1  const fs = require("fs");  
2  
3  function afterFileRead(err, contents) {  
4    console.log(contents);  
5  }  
6  
7  fs.readFile("a.txt", "utf-8", afterFileRead);  
8
```

Annotations below the code:

- Red box around `"a.txt"` with an arrow pointing to the word `string`.
- Red box around `"utf-8"` with an arrow pointing to the word `string`.
- Red box around `afterFileRead` with an arrow pointing to the word `function`.

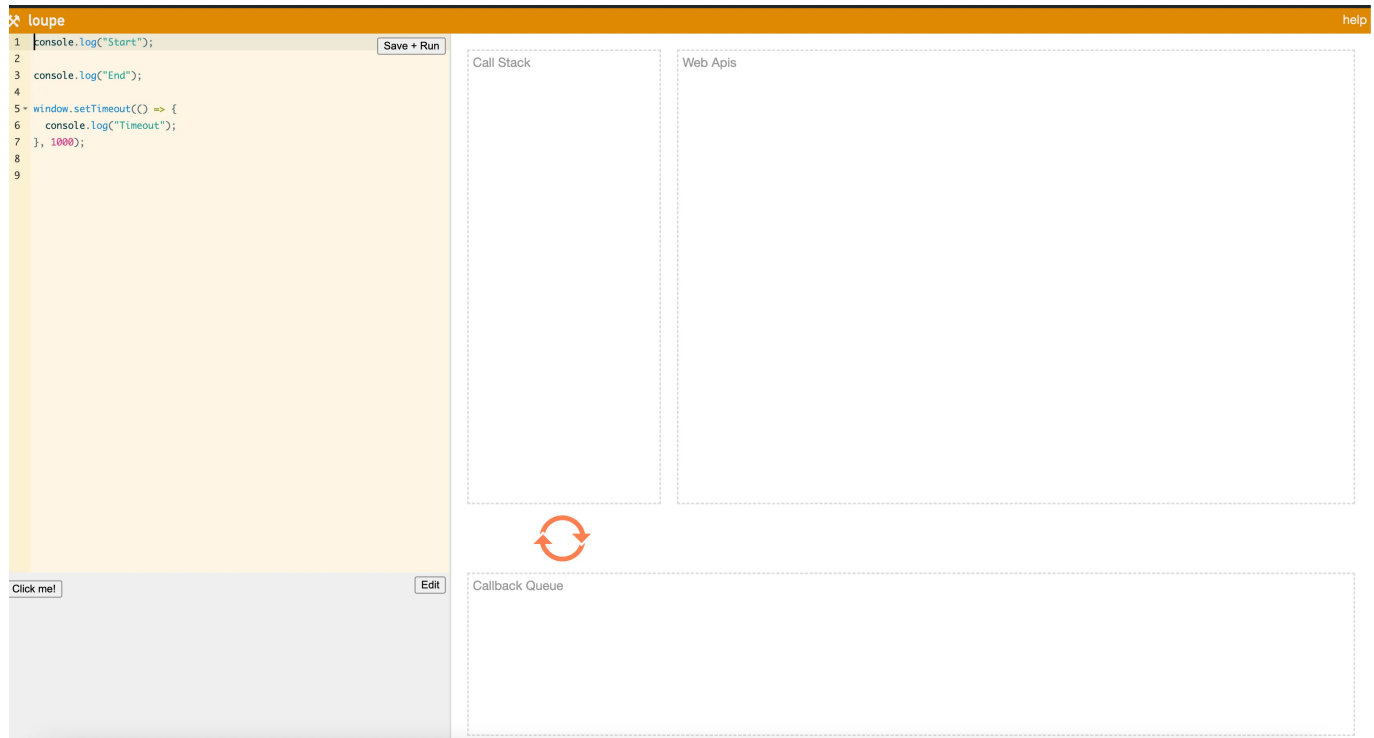
```
const fs = require("fs");  
  
fs.readFile("a.txt", "utf-8", function (err, contents) {  
  console.log(contents);  
});
```

- `setTimeout`
- `setTimeout` is another asynchronous function that executes a certain code after some time

```
function run() {  
  console.log("I will run after 1s");  
}  
  
setTimeout(run, 1000);  
console.log("I will run immediately");
```

JS Architecture for async code

How JS executes asynchronous code - <http://latentflip.com/loupe/>



- 1. Call Stack
 - The call stack is a data structure that keeps track of the function calls in your program. It operates in a "Last In, First Out" (LIFO) manner, meaning the last function that was called is the first one to be executed and removed from the stack.
 - When a function is called, it gets pushed onto the call stack. When the function completes, it's popped off the stack.

```
function first() {  
  console.log("First");  
}  
function second() {  
  first();  
  console.log("Second");  
}  
second();
```

- 2. Web APIs

- Web APIs are provided by the browser (or the Node.js runtime) and allow you to perform tasks that are outside the scope of the JavaScript language itself, such as making network requests, setting timers, or handling DOM events.
- 3. Callback Queue
 - The callback queue is a list of tasks (callbacks) that are waiting to be executed once the call stack is empty. These tasks are added to the queue by Web APIs after they have completed their operation.
- 4. Event loop
 - The event loop constantly checks if the call stack is empty. If it is, and there are callbacks in the callback queue, it will push the first callback from the queue onto the call stack for execution.

4 Promises and async, await

Aaj ghode khuleinge.



What we're doing today

- Classes in JS
- Revise callbacks
- Callback hell
- Promises
- Async await

- Assignments
- Will release a video on how to solve them
- <https://github.com/100xdevs-cohort-3/assignments/>

Classes in JS

- Primitive types
 - number
 - string
 - boolean
- Complex types
 - Objects
 - Arrays
- Classes
- In JavaScript, classes are a way to define blueprints for creating objects (these objects are different from the objects defined in the last section).
- For example

```
class Rectangle {
  constructor(width, height, color) {
    this.width = width;
    this.height = height;
    this.color = color;
  }

  area() {
    const area = this.width * this.height;
    return area;
  }

  paint() {
    console.log(`Painting with color ${this.color}`);
  }
}

const rect = new Rectangle(2, 4)
const area = rect.area();
console.log(area)
```

- Key Concepts
- Class Declaration:
 - You declare a class using the class keyword.

- Inside a class, you define properties (variables) and methods (functions) that will belong to the objects created from this class.
- Constructor:
 - A special method inside the class that is called when you create an instance (an object) of the class.
 - It's used to initialize the properties of the object.
- Methods:
 - Functions that are defined inside the class and can be used by all instances of the class.
- Inheritance:
 - Classes can inherit properties and methods from other classes, allowing you to create a new class based on an existing one.
- Static Methods:
 - Methods that belong to the class itself, not to instances of the class. You call them directly on the class.
- Getters and Setters:
 - Special methods that allow you to define how properties are accessed and modified.

Inheritance in classes

Inheritance in JavaScript classes allows one class to inherit properties and methods from another class. This mechanism enables code reuse, making it easier to create new classes that are based on existing ones, without having to duplicate code.

- Assignment #1 - Create a Circle class

```
class Circle {
  constructor(radius, color) {
    this.radius = radius;
    this.color = color;
  }

  area() {
    const area = this.radius * this.radius * Math.PI;
    return area;
  }

  paint() {
    console.log(`Painting with color ${this.color}`);
  }
}

const circle = new Circle(2, "red")
const area = circle.area();
console.log(area)
```

Can you see there is code repetition here and in the Rectangle class?

- Assignment #2 - Create a base shape class

- Base class

```
class Shape {
  constructor(color) {
    this.color = color;
  }

  paint() {
    console.log(`Painting with color ${this.color}`);
  }

  area() {
    throw new Error('The area method must be implemented in the subclass');
  }

  getDescription() {
    return `A shape with color ${this.color}`;
  }
}
```

- Rectangle class

```
class Rectangle extends Shape {
  constructor(width, height, color) {
    super(color); // Call the parent class constructor to set the color
    this.width = width;
    this.height = height;
  }

  area() {
    return this.width * this.height;
  }

  getDescription() {
    return `A rectangle with width ${this.width}, height ${this.height}, and color ${this.color}`;
  }
}
```

- Circle class

```
class Circle extends Shape {
  constructor(radius, color) {
    super(color); // Call the parent class constructor to set the color
    this.radius = radius;
  }
}
```



```
    }

    area() {
        return Math.PI * this.radius * this.radius;
    }

    getDescription() {
        return `A circle with radius ${this.radius} and color
        ${this.color}`;
    }
}
```

- Try playing with it

```
const circle = new Circle(20);
console.log(circle.area());
```

Some more classes

- Date

```
const now = new Date(); // Current date and time
console.log(now.toISOString()); // Outputs the date in ISO format
```

- Maps

```
const map = new Map();
map.set('name', 'Alice');
map.set('age', 30);
console.log(map.get('name'));
```

Promise class



- Calling a promise is easy, defining your own promise is where things get hard
- A Promise in JavaScript is an object that represents the eventual completion (or failure) of an asynchronous operation and its resulting value. Promises are used to handle asynchronous operations more effectively than traditional callback functions, providing a cleaner and more manageable way to deal with code that executes asynchronously, such as API calls, file I/O, or timers.
- Using a function that returns a promise
 - Ignore the function definition of `setTimeoutPromisified` for now

```
function setTimeoutPromisified(ms) {  
  return new Promise(resolve => setTimeout(resolve, ms));  
}  
  
function callback() {  
  console.log("3 seconds have passed");  
}  
  
setTimeoutPromisified(3000).then(callback)
```

Promisified version

```
setTimeoutPromisified(3000).then(callback)
```

Callback version

```
setTimeout(callback, 3000);
```

Callback hell

- Q: Write code that
 - logs hi after 1 second
 - logs hello 3 seconds after step 1
 - logs hello there 5 seconds after step 2
 - Solution (has callback hell)

```
setTimeout(function () {  
  console.log("hi");  
  setTimeout(function () {  
    console.log("hello");  
  
    setTimeout(function () {  
      console.log("hello there");  
    }, 5000);  
  }, 3000);  
, 1000);
```

- Alt solution (doesn't really have callback hell)

```
function step3Done() {
  console.log("hello there");
}

function step2Done() {
  console.log("hello");
  setTimeout(step3Done, 5000);
}

function step1Done() {
  console.log("hi");
  setTimeout(step2Done, 3000);
}

setTimeout(step1Done, 1000);
```

- Promisified version
 - Now use the promisified version we saw in the last slide

```
function setTimeoutPromisified(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

- Solution #1 (has callback hell)

```
function setTimeoutPromisified(ms) {
  return new Promise((resolve) => setTimeout(resolve, ms));
}

setTimeoutPromisified(1000).then(function () {
  console.log("hi");
  setTimeoutPromisified(3000).then(function () {
    console.log("hello");
    setTimeoutPromisified(5000).then(function () {
      console.log("hello there");
    });
  });
});
```

- Alt solution

```
setTimeoutPromisified(1000)
  .then(function () {
    console.log("hi");
    return setTimeoutPromisified(3000);
  });
```

```
    })
    .then(function () {
      console.log("hello");
      return setTimeoutPromisified(5000);
    })
    .then(function () {
      console.log("hello there");
    });
  });
```

Async await syntax

- The async and await syntax in JavaScript provides a way to write asynchronous code that looks and behaves like synchronous code, making it easier to read and maintain.
- It builds on top of Promises and allows you to avoid chaining .then() and .catch() methods while still working with asynchronous operations.
- async/await is essentially syntactic sugar on top of Promises.
- Assignment
 - Write code that
 - logs hi after 1 second
 - logs hello 3 seconds after step 1
 - logs hello there 5 seconds after step 2

```
function setTimeoutPromisified(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function solve() {
  await setTimeoutPromisified(1000);
  console.log("hi");
  await setTimeoutPromisified(3000);
  console.log("hello");
  await setTimeoutPromisified(5000);
  console.log("hi there");
}

solve();
```

- Things to keep in mind
 - You can only call await inside a function if that function is async
 - You cant have a top level await

Defining your own async function

- Q: Write a function that
 - Reads the contents of a file
 - Trims the extra space from the left and right
 - Writes it back to the file

- 1. Callback approach In the callback approach, the function signature should look something like this

```
function onDone() {  
    console.log("file has been cleaned");  
}  
cleanFile("a.txt", onDone)
```

- Solution

```
const fs = require("fs");  
function cleanFile(filePath, cb) {  
    fs.readFile(filePath, "utf-8", function (err, data) {  
        data = data.trim();  
        fs.writeFile(filePath, data, function () {  
            cb();  
        });  
    });  
}  
  
function onDone() {  
    console.log("file has been cleaned");  
}  
cleanFile("a.txt", onDone);
```

- 2. Promisified approach In the promisified approach, the function signature should look something like this

```
async function main() {  
    await cleanFile("a.txt")  
    console.log("Done cleaning file");  
}  
  
main();
```

- Solution

```
const fs = require("fs");  
function cleanFile(filePath, cb) {  
    return new Promise(function (resolve) {  
        fs.readFile(filePath, "utf-8", function (err, data) {  
            data = data.trim();  
            fs.writeFile(filePath, data, function () {  
                resolve();  
            });  
        });  
    });  
};
```

```
});  
}  
  
async function main() {  
  await cleanFile("a.txt");  
  console.log("Done cleaning file");  
}  
  
main();
```

err first callback vs rejects in promises

- Callbacks
 - fs.readFile function used an err first callback approach to propagate back errors

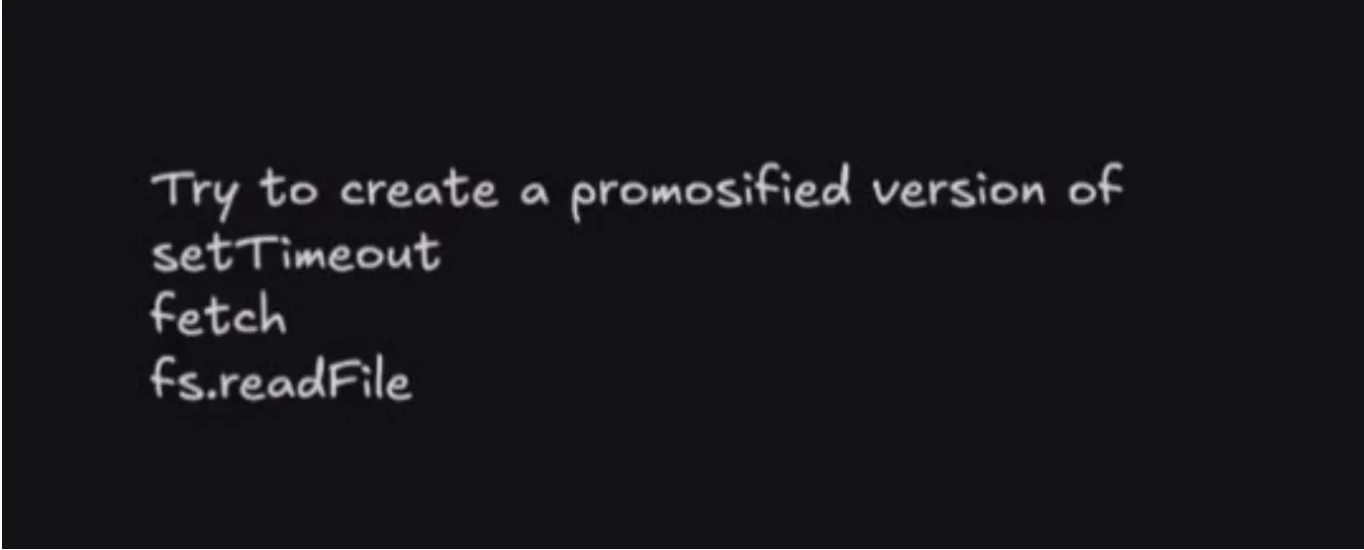
```
const fs = require("fs")  
function afterDone(err, data) {  
  if (err) {  
    console.log("Error while reading file");  
  } else {  
    console.log(data)  
  }  
}  
  
fs.readFile("a.txt", "utf-8", afterDone);
```

- Promises
 - Promises use the reject argument to propagate errors

```
const fs = require("fs");  
  
function readFilePromisified(filePath) {  
  return new Promise(function (resolve, reject) {  
    fs.readFile(filePath, "utf-8", function (err, data) {  
      if (err) {  
        reject("Error while reading file");  
      } else {  
        resolve(data);  
      }  
    });  
  });  
}  
  
function onDone(data) {  
  console.log(data);  
}  
  
function onError(err) {  
  console.log("Error: " + err);  
}
```

```
}  
  
readFilePromisified("a.txt").then(onDone).catch(onError);
```

Assignment



Try to create a promosified version of
setTimeout
fetch
fs.readFile